

Тема 7.2 Процессор контроллера

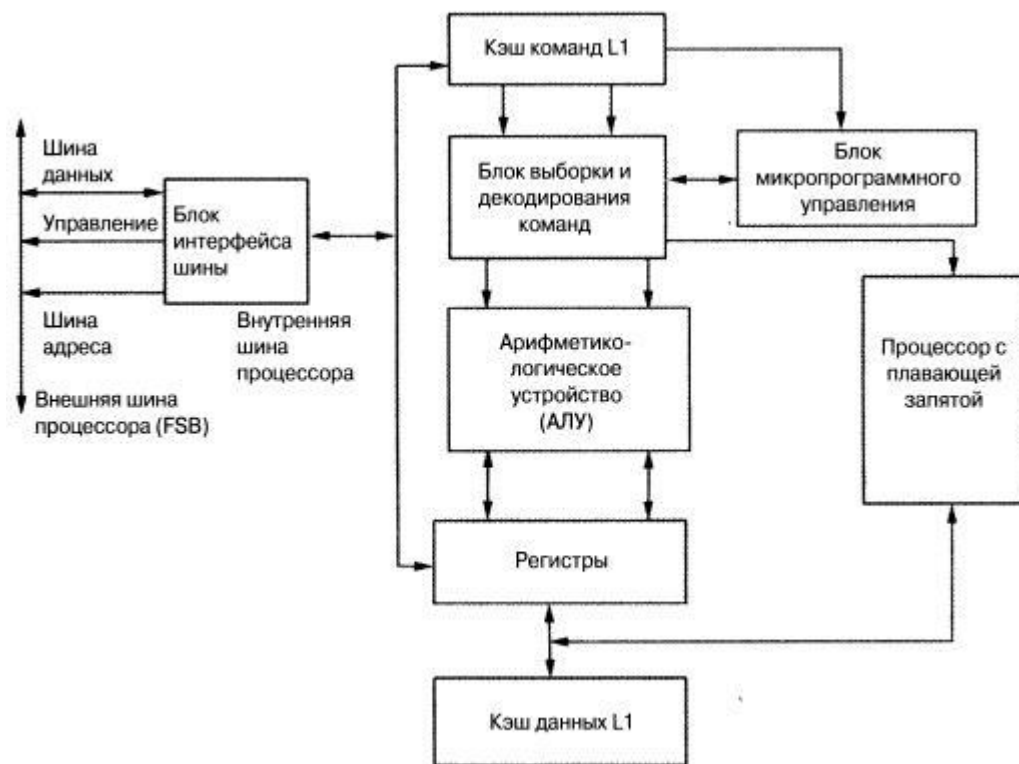
Назначение процессора. Архитектурные варианты процессора

Центральный процессор (ЦП или центральное процессорное устройство - ЦПУ; англ. central processing unit, CPU, дословно — центральное вычислительное устройство) — исполнитель машинных инструкций, часть аппаратного обеспечения компьютера или программируемого логического контроллера, отвечающий за выполнение операций, заданных программами.

Центральный процессор в общем случае содержит в себе:

- арифметико-логическое устройство;
- шины данных и шины адресов;
- регистры;
- счетчики команд;
- кэш — очень быструю память малого объема (от 8 до 512 Кбайт);
- математический сопроцессор чисел с плавающей точкой.

Упрощенная блок-схема центрального процессора Р6 изображена на рисунке ниже:



Упрощенная блок-схема центрального процессора Р6

АРХИТЕКТУРА ФОН НЕЙМАНА

Большинство современных процессоров для персональных компьютеров в общем основаны на той или иной версии

циклического процесса последовательной обработки информации, изобретённого Джоном фон Нейманом.

Д. фон Нейман придумал схему постройки компьютера в 1946 году.

Важнейшие этапы этого процесса приведены ниже. В различных архитектурах и для различных команд могут потребоваться дополнительные этапы. Например, для арифметических команд могут потребоваться дополнительные обращения к памяти, во время которых производится считывание операндов и запись результатов. Отличительной особенностью архитектуры фон Неймана является то, что инструкции и данные хранятся в одной и той же памяти.

Этапы цикла выполнения:

1. Процессор выставляет число, хранящееся в регистре счётчика команд, на шину адреса, и отдаёт памяти команду чтения;
2. Выставленное число является для памяти адресом; память, получив адрес и команду чтения, выставляет содержимое, хранящееся по этому адресу, на шину данных, и сообщает о готовности;
3. Процессор получает число с шины данных, интерпретирует его как команду (машинную инструкцию) из своей системы команд и исполняет её;
4. Если последняя команда не является командой перехода, процессор увеличивает на единицу (в предположении, что длина каждой команды равна единице) число, хранящееся в счётчике команд; в результате там образуется адрес следующей команды;
5. Снова выполняется п. 1.

Данный цикл выполняется неизменно, и именно он называется *процессом* (откуда и произошло название устройства).

Во время процесса процессор считывает последовательность команд, содержащихся в памяти, и исполняет их. Такая последовательность команд называется программой и представляет алгоритм работы процессора. Очерёдность считывания команд изменяется в случае, если процессор считывает команду перехода — тогда адрес следующей команды может оказаться другим. Другим примером изменения процесса может служить случай получения команды останова или переключение в режим обработки прерывания.

Команды центрального процессора являются самым нижним уровнем управления компьютером, поэтому выполнение каждой команды неизбежно и безусловно. Не производится никакой проверки на допустимость выполняемых действий, в частности, не проверяется возможная потеря ценных данных. Чтобы компьютер выполнял только допустимые действия, команды должны быть соответствующим образом организованы в виде необходимой программы.

Скорость перехода от одного этапа цикла к другому определяется тактовым генератором. Тактовый генератор вырабатывает импульсы,

служащие ритмом для центрального процессора. Частота тактовых импульсов называется тактовой частотой.

КОНВЕЙЕРНАЯ АРХИТЕКТУРА

Конвейерная архитектура (*pipelining*) была введена в центральный процессор с целью повышения быстродействия. Обычно для выполнения каждой команды требуется осуществить некоторое количество однотипных операций, например: выборка команды из ОЗУ, дешифрация команды, адресация операнда в ОЗУ, выборка операнда из ОЗУ, выполнение команды, запись результата в ОЗУ. Каждую из этих операций сопоставляют одной ступени конвейера. Например, конвейер микропроцессора с архитектурой MIPS-I содержит четыре стадии:

- получение и декодирование инструкции (Fetch)
- адресация и выборка операнда из ОЗУ (Memory access)
- выполнение арифметических операций (Arithmetic Operation)
- сохранение результата операции (Store)

После освобождения k -й ступени конвейера она сразу приступает к работе над следующей командой. Если предположить, что каждая ступень конвейера тратит единицу времени на свою работу, то выполнение команды на конвейере длиной в n ступеней займёт n единиц времени, однако в самом оптимистичном случае результат выполнения каждой следующей команды будет получаться через каждую единицу времени.

Действительно, при отсутствии конвейера выполнение команды займёт n единиц времени (так как для выполнения команды по прежнему необходимо выполнять выборку, дешифрацию и т. д.), и для исполнения m команд понадобится $n \cdot m$ единиц времени; при использовании конвейера (в самом оптимистичном случае) для выполнения m команд понадобится всего лишь $n + m$ единиц времени.

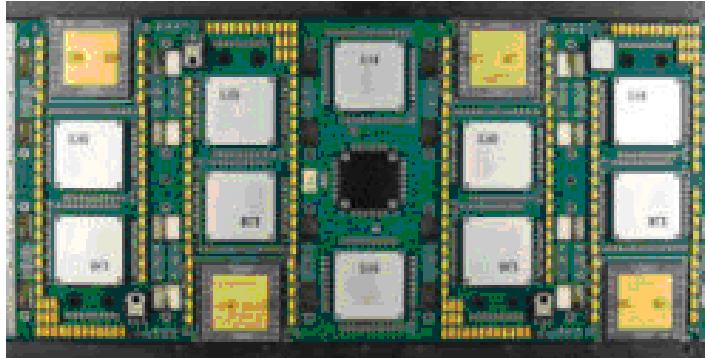
Факторы, снижающие эффективность конвейера:

- простой конвейера, когда некоторые ступени не используются (напр., адресация и выборка операнда из ОЗУ не нужны, если команда работает с регистрами);
- ожидание: если следующая команда использует результат предыдущей, то последняя не может начать выполняться до выполнения первой (это преодолевается при использовании внеочередного выполнения команд, out-of-order execution);
- очистка конвейера при попадании в него команды перехода (эту проблему удаётся сгладить, используя предсказание переходов).

Некоторые современные процессоры имеют более 30 ступеней в конвейере, что увеличивает производительность процессора, однако приводит к большому времени простоя (например, в случае ошибки в предсказании условного перехода).

СУПЕРСКАЛЯРНАЯ АРХИТЕКТУРА

Суперскалярность — архитектура вычислительного ядра, использующая несколько декодеров команд, которые могут нагружать работой множество исполнительных блоков. Планирование исполнения потока команд является динамическим и осуществляется самим вычислительным ядром.



Плата CRAY T3E-136/ac

Если в процессе работы команды, обрабатываемые конвейером, не противоречат друг другу, и одна не зависит от результата другой, то такое устройство может осуществить параллельное выполнение команд. В суперскалярных системах решение о запуске инструкции на исполнение принимает сам вычислительный модуль, что требует много ресурсов. В более поздних системах, таких как Эльбрус-3 и Itanium, используется статпланирование, то есть параллельные инструкции объединяются компилятором в длинную команду, в которой все инструкции заведомо параллельные (архитектура VLIW).

Процессоры, поддерживающие суперскалярность:

- Pentium, AMD Duron, AMD ATHLON и другие более поздние процессоры с архитектурой x86
- Последние процессоры с архитектурами SPARC, ARM, MIPS

CISC-ПРОЦЕССОРЫ

CISC (англ. *Complex Instruction Set Computing*) — концепция проектирования процессоров, которая характеризуется следующим набором свойств:

- Нефиксированным значением длины команды.
- Арифметические действия кодируются в одной инструкции.
- Небольшим числом регистров, каждый из которых выполняет строго определённую функцию.

Типичными представителями являются процессоры на основе x86 команд (исключая современные Intel Pentium 4, Pentium D, Core, AMD Athlon, Phenom, которые являются гибридными) и процессоры Motorola MC680x0.

Наиболее распространённая архитектура современных настольных, серверных и мобильных процессоров построена по архитектуре Intel x86 (или x86-64 в случае 64-разрядных процессоров). Формально, все x86-процессоры являлись CISC-процессорами, однако новые процессоры, начиная с Intel486DX, являются CISC-процессорами с RISC-ядром. Они непосредственно перед исполнением преобразуют CISC-инструкции процессоров x86 в более простой набор внутренних инструкций RISC. В микропроцессор встраивается аппаратный транслятор, превращающий команды x86 в команды внутреннего RISC-процессора. При этом одна команда x86 может порождать несколько RISC-команд (в случае процессоров типа P6 — до 4-х RISC команд в большинстве случаев). Исполнение команд происходит на суперскалярном конвейере одновременно по несколько штук. Это потребовалось для увеличения скорости обработки CISC-команд, так как известно, что любой CISC-процессор уступает RISC-процессорам по количеству выполняемых операций в секунду. В итоге, такой подход и позволил поднять производительность CPU.

RISC-ПРОЦЕССОРЫ

RISC (англ. *Reduced Instruction Set Computer*; неправильно — *Reduced Instruction Set*

Computing) — компьютер с сокращённым набором команд. Архитектура процессоров, построенная на основе сокращённого набора команд. Характеризуется наличием команд фиксированной длины, большого количества регистров, операций типа регистр-регистр, а также отсутствием косвенной адресации.

Это концепция проектирования процессоров (ЦПУ), которая во главу ставит следующий принцип: более компактные и простые инструкции выполняются быстрее. Простая архитектура позволяет удешевить процессор, поднять тактовую частоту, а также распараллелить исполнение команд между несколькими блоками исполнения (т. н. суперскалярные архитектуры процессоров). Многие ранние RISC-процессоры даже не имели команд умножения и деления. Идея создания RISC процессоров пришла после того, как в 1970-х годах ученые из IBM обнаружили, что многие из функциональных особенностей традиционных ЦПУ игнорировались программистами. Отчасти это был побочный эффект сложности компиляторов. В то время компиляторы могли использовать лишь часть из набора команд процессора. Следующее открытие заключалось в том, что, поскольку некоторые сложные операции использовались редко, они как правило были медленнее, чем те же действия, выполняемые набором простых команд. Это происходило из-за того, что создатели процессоров тратили гораздо меньше времени на улучшение сложных команд, чем на улучшение простых.

Характерные особенности RISC-процессоров:

- Фиксированная длина машинных инструкций (например, 32 бита) и простой формат команды.

- Специализированные команды для операций с памятью — чтения или записи. Операции вида «прочитать-изменить-записать» отсутствуют. Любые операции «изменить» выполняются только над содержимым регистров (т. н. load-and-store архитектура).
- Большое количество регистров общего назначения (32 и более).
- Отсутствие поддержки операций вида «изменить» над укороченными типами данных — байт, 16-битное слово. Так, например, система команд DEC Alpha содержала только операции над 64-битными словами, и требовала разработки и последующего вызова процедур для выполнения операций над байтами, 16- и 32-битными словами.
- Отсутствие микропрограмм внутри самого процессора. То, что в CISC процессоре выполняется микропрограммами, в RISC процессоре выполняется как обыкновенный (хотя и помещенный в специальное хранилище) машинный код, не отличающийся принципиально от кода ядра ОС и приложений. Так, например, обработка отказов страниц в DEC Alpha и интерпретация таблиц страниц содержалась в так называемом PALCode (Privileged Architecture Library), помещенном в ПЗУ. Заменой PALCode можно было превратить процессор Alpha из 64-битного в 32-битный, а также изменить порядок байт в слове и формат входов таблиц страниц виртуальной памяти.

Первые RISC-процессоры были разработаны в начале 1980-х годов в Стэнфордском и Калифорнийском университетах США. Концепция RISC разработана Джоном Коком (John Cocke) из IBM Research, название придумано Дэвидом Паттерсоном (David Patterson). Они выполняли небольшой (50–100) набор команд, тогда как обычные CISC (Complex Instruction Set Computer) выполняли 100—200. Среди первых реализаций этой архитектуры были процессоры MIPS, PowerPC, SPARC, Alpha, PA-RISC. В мобильных устройствах широко используются ARM-процессоры.

MISC-ПРОЦЕССОРЫ

MISC (англ. *Minimal Instruction Set Computer*) — процессор, работающий с минимальным набором длинных команд. Дальнейшее развитие идей команды Чака Мура, который полагает, что принцип простоты, изначальный для RISC-процессоров, слишком быстро отошёл на задний план. В пылу борьбы за максимальное быстродействие, RISC догнал и перегнал многие CISC процессоры по сложности. Архитектура MISC строится на стековой вычислительной модели с ограниченным числом команд (примерно 20-30 команд).

Увеличение разрядности процессоров привело к идее укладки нескольких команд в одно большое слово. Это позволило использовать возросшую производительность компьютера и его возможность обрабатывать одновременно несколько потоков данных. Кроме этого MISC использует

стековую модель вычислительного устройства и основные команды работы со стеком Forth языка. MISC принцип может лежать в основе микропрограммы выполнения Java и .Net программ, хотя по количеству используемых команд они нарушают принцип MISC

Процессоры, образующие «компьютеры с минимальным набором команд» MISC, как и процессоры RISC, характеризуются небольшим числом чаще всего встречающихся команд. Вместе с этим, принцип «очень длинных слов команд» VLIW обеспечивает выполнение группы непротиворечивых команд за один цикл работы процессора. Порядок выполнения команд распределяется таким образом, чтобы в максимальной степени загрузить маршруты, по которым проходят потоки данных. Таким образом архитектура MISC объединила вместе суперскалярную и VLIW концепции. Компоненты процессора просты и работают с высокими скоростями.

Устройство и принцип действия процессора

Принцип работы центрального процессора можно описать следующим образом

Данные для обработки под управлением сигналов на шине управления и адреса поступают из ОЗУ через блок интерфейса шины в кэш данных процессора, а команды — в кэш-память команд.

На этом этапе фаза выборки команды (Fetch Instruction) переходит в фазу декодирования команды (Decode). Команда декодируется и полученная инструкция передается в блок микропрограммного управления (МПУ).

МПУ представляет собой перепрограммируемое запоминающее устройство, в котором хранятся цепочки микрокоманд логики управления, т.е. алгоритмы выполнения всех инструкций процессора данной архитектуры.

В этом блоке код инструкции «раскрывается», в результате чего генерируются сигналы управления, настраивающие пути данных для реализации фазы выбора операндов (Operands) и выполнения команды (Execution).

Операнды доставляются для вычислений из различных источников — внешних устройств, ОЗУ, а также из регистров процессора. Коды режимов адресации ячеек ОЗУ или явные адреса операндов указаны в одном из полей текущей команды.

Регистры процессора представлены несколькими группами, в частности процессор содержит регистры общего назначения процессора (РОН), регистры сегментов, регистры указателя команд и флагов, системные регистры. Регистры называются также сверхоперативным запоминающим устройством компьютера (СОЗУ). СОЗУ содержит небольшое количество ячеек памяти, предназначенных для формирования текущего адреса, сегментирования памяти, хранения компонентов защищенного режима, хранения операндов и результатов вычислений, подсчета операций, организации стека и реализации прочих сервисных и системных функций.

Арифметические и логические вычисления выполняются над операндами в арифметико-логическом блоке (АЛУ), а высокоточные вычисления очень малых или очень больших величин выполняются в процессоре с плавающей запятой (ППЗ).

ППЗ — отдельный процессор, который наряду с центральным процессором интегрирован в кристалл микропроцессора для повышения производительности и увеличения точности вычислений особых функций. Он может работать с целыми, десятичными, коротким и длинными вещественными числами. ПЗУ имеет собственную систему команд. Он содержит свое АЛУ, МПУ и располагает отдельными РОН. Но он не может выбирать команды — за него это делает центральный процессор.

Итогом вычислений является формирование кода состояния и результата. Оба этих информационных блока пересылаются на хранение в регистры. Здесь же в соответствии с указанным в текущей команде режимом адресации формируется адрес, по которому передается результат вычислений. Это может быть ячейка ОЗУ или регистр интеллектуального внешнего устройства.

Скорость работы процессора зависит от нескольких факторов — от тактовой частоты процессора, пропускной способности внутренних и внешних шин, разрядности регистров и архитектурных особенностей процессора.

Арифметико-логическое устройство (АЛУ)

предназначено для выполнения арифметических и логических операций преобразования информации.

Функционально в простейшем варианте АЛУ состоит из двух регистров, сумматора и схем управления (местного устройства управления).



Все регистры можно разделить на четыре группы:

1. **универсальные регистры**: AX, BX, CX, DX;
2. **сегментные регистры**: CS, DS, SS, ES;
3. **регистры смещения**: IP, SP, BP, SI, DI;
4. **регистр флагов**: FL.

регистр BX часто используется для хранения адреса базы в сегменте данных и начального адреса поля памяти при работе с массивами;

регистр CX — регистр-счетчик, используется как счетчик числа повторений при циклических операциях;

регистр DX используется как расширение регистра-аккумулятора при работе с 32-разрядными числами и при выполнении операций умножения и деления, используется для хранения номера порта при операциях ввода-вывода и т. д.

Регистры сегментной адресации CS, DS, SS, ES используются для хранения начальных адресов полей памяти (сегментов), отведенных в программах для хранения:

- **команд программы** (сегмент кода — CS);
- **данных** (сегмент данных — DS);
- **стековой области памяти** (сегмент стека — SS);
- **дополнительной области памяти данных** при межсегментных пересылках (расширенный сегмент — ES), поскольку размер сегмента в реальном режиме работы МП ограничен величиной 64 Кбайт.

Регистры смещений (внутрисегментной адресации) IP, SP, BP, SI, DI предназначены для хранения относительных адресов ячеек памяти внутри сегментов (смещений относительно начала сегментов):

- **регистр IP** (Instruction Pointer) - смещение адреса текущей команды программы;